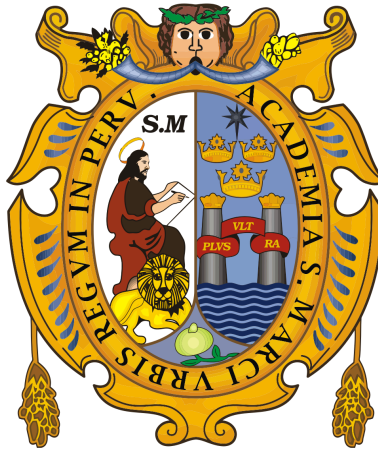


“Año de la Esperanza y el Fortalecimiento de la Democracia”

Universidad Nacional Mayor de San Marcos

(Universidad del Perú. Decana de América)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



Curso:

Métodos Numéricos

Docente:

Josué Miranda

Tema:

Evaluación y reconstrucción de señales en sistemas de procesamiento digital

Integrantes:

Flores Carrion, Harol kenyi - 22190133

Bellido Araujo, Alvaro Daniel – 23190302

Izquierdo Cabrel, Bryan Jair – 21190057

Lima, Perú

Evaluación y reconstrucción de señales en sistemas de procesamiento digital mediante interpolación por splines cúbicos

1. Introducción y Justificación Experimental

Las señales analógicas están presentes en numerosos sistemas de ingeniería, como sensores industriales, equipos biomédicos, sistemas de telecomunicaciones e instrumentos de medición. Sin embargo, los sistemas digitales modernos requieren convertir estas señales continuas en secuencias discretas mediante un proceso conocido como muestreo. La correcta adquisición y reconstrucción de señales es fundamental para garantizar que la información original pueda ser procesada, analizada y visualizada sin pérdidas significativas.

En este proyecto se implementa un arreglo experimental basado en un Arduino UNO y MATLAB para adquirir muestras de una señal senoidal generada experimentalmente mediante la salida de audio de un teléfono móvil. Posteriormente, las muestras discretas son procesadas mediante un método numérico de interpolación —trazadores cúbicos (splines)— con el fin de reconstruir una aproximación continua de la señal original y estimar su valor en instantes de tiempo no muestreados.

La señal generada por el teléfono oscila entre -0.5 V y 0.5 V , un rango que dañaría la entrada analógica del Arduino si se conectara directamente. Por ello se implementó un circuito de acondicionamiento compuesto por un divisor de voltaje resistivo ($R_1 = R_2 = 1\text{ k}\Omega$) y un capacitor de acoplo de $10\text{ }\mu\text{F}$, que desplaza la señal a un nivel de offset seguro (aproximadamente 0 V a 1 V) sin alterar su componente de interés, permitiendo su digitalización mediante el conversor analógico-digital (ADC) de 10 bits del Arduino en el pin A0.

El estudio de este proceso permite comprender conceptos fundamentales de señales y sistemas, tales como el teorema de muestreo de Nyquist, el aliasing, la cuantización y la reconstrucción de señales mediante interpolación numérica. Estos principios constituyen la base de numerosas aplicaciones tecnológicas, incluyendo osciloscopios digitales, sistemas de adquisición de datos (DAQ), procesamiento de audio, comunicaciones digitales y equipos de instrumentación electrónica utilizados en la industria y la investigación. Comprender cómo un método numérico de interpolación reconstruye información faltante entre muestras discretas es, además, una habilidad transferible a cualquier problema de ingeniería donde los datos experimentales sean inherentemente discretos.

2. Esquema y Fotografía del Arreglo Experimental

El sistema experimental sigue el flujo mostrado en la Figura 1: la señal analógica generada por el teléfono es acondicionada, digitalizada por el Arduino y enviada por puerto serial a MATLAB, donde se procesa mediante interpolación por splines cúbicos y finalmente se visualiza.

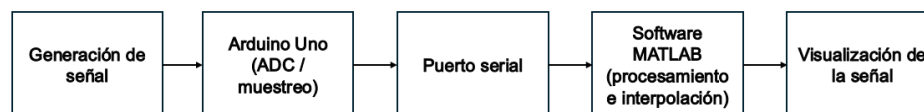


Figura 1. Diagrama de bloques del sistema.

La Figura 2 muestra el esquemático del circuito de acondicionamiento: un divisor de voltaje resistivo ($R_1, R_2 = 1\text{ k}\Omega$) fija el punto de offset a la mitad de la alimentación de 5 V , mientras que el capacitor de acoplo C_1 ($10\text{ }\mu\text{F}$) bloquea la componente de continua (DC) de la fuente y permite el paso de la componente variable de la señal de audio, sumando ambas para mantener la salida dentro del rango $0\text{--}1\text{ V}$ que exige la entrada analógica del Arduino.

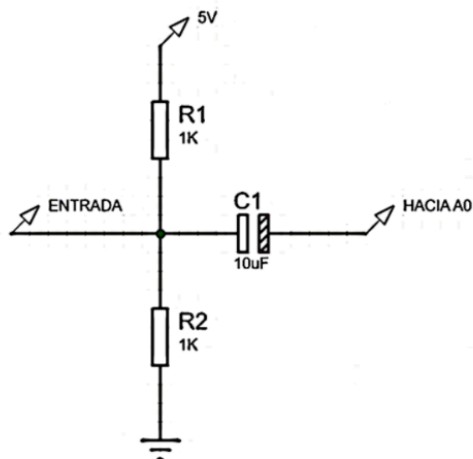


Figura 2. Esquemático del circuito divisor de voltaje con acoplo capacitivo.

La Figura 3 muestra la implementación física del arreglo: el Arduino UNO conectado al protoboard donde se ubican las resistencias del divisor de voltaje y el capacitor de acoplo, con la salida del circuito conectada a la entrada analógica A0.

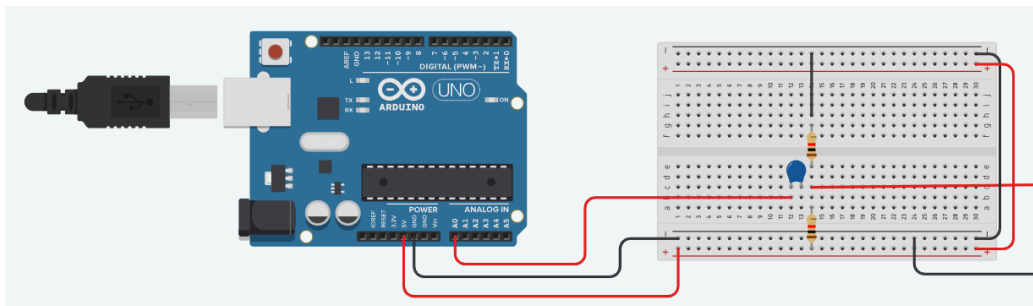


Figura 3. Implementación física del circuito sobre protoboard, conectado al Arduino UNO.

3. Datos Medidos e Incertidumbre

Las muestras se adquirieron a una frecuencia de muestreo fija $F_s = 4$ Hz (una muestra cada 250 ms, controlada por temporizador del microcontrolador), tomando 21 muestras a lo largo de 5 segundos. El firmware del Arduino envía cada lectura del ADC ya convertida a voltios mediante la relación $\text{voltaje} = \text{valorADC} \times (5.0 / 1023.0)$.

Incertidumbres consideradas:

- Resolución del ADC (10 bits, referencia de 5 V): $\Delta V = 5 \text{ V} / 1023 \approx 4.89 \text{ mV}$. Al tomar el error de cuantización como $\pm \frac{1}{2} \text{ LSB}$, la incertidumbre de cada lectura de voltaje es $\pm 0.0025 \text{ V}$.
- Base de tiempo: el intervalo entre muestras está definido por el temporizador interno del microcontrolador (función `delay()`), cuya desviación típica es del orden de partes por millón; se asume una incertidumbre despreciable de $\pm 0.001 \text{ s}$.
- No se incluye incertidumbre adicional del instrumento de medida externo, ya que la adquisición es enteramente digital (ADC embebido), a diferencia de un voltímetro o termómetro manual.

La Tabla 1 presenta las 21 muestras de voltaje adquiridas por el circuito físico, con su tiempo correspondiente.

Tabla 1. Datos experimentales de Voltaje vs. Tiempo (circuito físico).

Tiempo ($t \pm 0.001$ s)	Voltaje ($V \pm 0.0025$ V)
0.00	0.7038
0.25	0.8993
0.50	0.9531
0.75	0.7185
1.00	0.7918
1.25	0.9824
1.50	0.7771
1.75	0.7136
2.00	0.9629
2.25	0.8847
2.50	0.6843
2.75	0.8700
3.00	0.9531
3.25	0.7087
3.50	0.7771
3.75	0.9775
4.00	0.7869
4.25	0.7136
4.50	0.9433
4.75	0.8749
5.00	0.6794

4. Procesamiento de Datos mediante Métodos Numéricos

4.1. Interpolación por Splines Cúbicos (Reconstrucción de la señal)

Objetivo:

Reconstruir una aproximación continua de la señal analógica original a partir de las 21 muestras discretas adquiridas, y estimar el voltaje en instantes de tiempo no muestreados (por ejemplo, entre dos muestras consecutivas).

Justificación del método:

Se eligió la interpolación por trazadores cúbicos (splines) en lugar de un polinomio de interpolación global (Lagrange o Newton de grado 20) porque un polinomio de grado alto ajustado a 21 puntos tiende a oscilar de forma no física entre los nodos (fenómeno de Runge), especialmente en señales con variaciones rápidas como la senoidal muestreada en este proyecto. El spline cúbico, al construir un polinomio de tercer grado distinto en cada subintervalo con continuidad de valor, primera y segunda derivada en los nodos, produce una curva suave y numéricamente estable, adecuada para aproximar el comportamiento de una señal periódica entre muestras.

Método aplicado:

Se utilizó la función `spline()` de MATLAB (trazador cúbico con condición de frontera "not-a-knot") sobre el vector de tiempo `t` y el vector de muestras adquiridas, generando un vector de tiempo interpolado con 20 veces mayor resolución (`linspace`) para visualizar la curva reconstruida junto con las muestras originales.

Código MATLAB (fragmento relevante):

```
% Interpolación por Splines Cúbicos
t_interp = linspace(min(t), max(t), length(t)*20);
muestras_interp = spline(t, muestras, t_interp);
```

Resultado obtenido (ejemplo de estimación en tiempo no tabulado):

Para ilustrar la capacidad de reconstrucción del método, se estimó el voltaje en $t = 1.10$ s, un instante no muestreado ubicado entre las muestras $t = 1.00$ s (0.7918 V) y $t = 1.25$ s (0.9824 V). El spline cúbico entrega:

$$V(1.10 \text{ s}) \approx 0.8939 \text{ V}$$

Este valor es consistente con la tendencia creciente observada entre ambas muestras vecinas, lo que confirma que el spline preserva la forma local de la señal sin introducir oscilaciones espurias.

Las Figuras 4 y 5 muestran, respectivamente, las 21 muestras adquiridas (señal muestreada) y la comparación entre dichas muestras y la curva reconstruida mediante el spline cúbico.

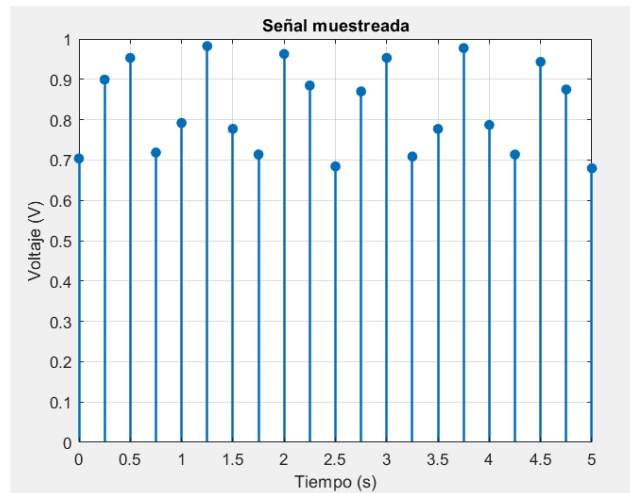


Figura 4. Señal muestreada: 21 valores de voltaje adquiridos a $F_s = 4$ Hz.

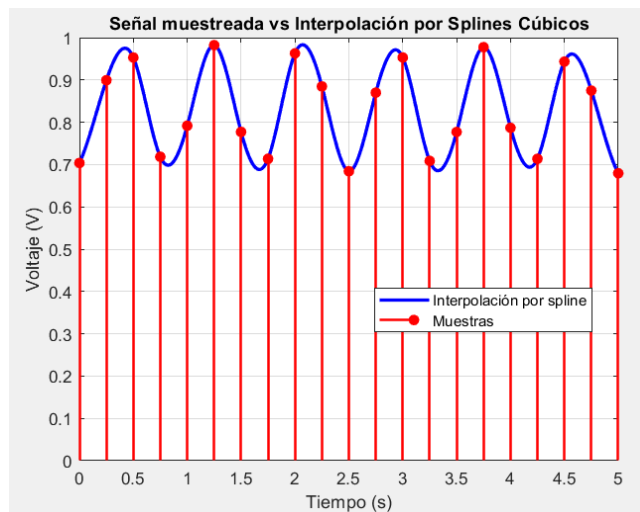


Figura 5. Señal muestreada (rojo) vs. reconstrucción por spline cúbico (azul).

4.2. Derivación Numérica (Cálculo de Tasas de Cambio)

Objetivo

Estimar la tasa de cambio instantánea del voltaje, dV/dt , a partir de las 21 muestras discretas adquiridas, con el fin de caracterizar la velocidad de variación de la señal en cada instante de muestreo, sin necesidad de conocer una expresión analítica de $V(t)$.

Justificación del método

Se optó por el método de diferencias finitas centradas para los puntos internos de la señal, dado que ofrece un error de truncamiento de orden $O(h^2)$, superior a las diferencias hacia adelante o hacia atrás simples, que son de orden $O(h)$. En los extremos del intervalo ($t = 0$ y $t = 5$ s), donde no se dispone de un punto vecino en ambos lados, se emplearon diferencias hacia adelante y hacia atrás, respectivamente, como es habitual en la práctica numérica cuando el dominio de datos es finito. Esta combinación de esquemas permite mantener una precisión consistente en todo el intervalo, evitando degradar la exactitud únicamente en los bordes.

Método aplicado

Se implementó el siguiente esquema de diferenciación numérica:

- Extremo inicial (diferencia hacia adelante): $dV/dt(t_1) = [V(t_2) - V(t_1)] / \Delta t$
- Puntos internos (diferencia centrada): $dV/dt(t_i) = [V(t_{i+1}) - V(t_{i-1})] / (2\Delta t)$
- Extremo final (diferencia hacia atrás): $dV/dt(t_N) = [V(t_N) - V(t_{N-1})] / \Delta t$

con un paso de muestreo constante $\Delta t = 0.25$ s.

Código MATLAB (fragmento relevante)

```
matlab
% Derivación Numérica por Diferencias Finitas
dV_dt(1) = (V(2) - V(1)) / dt;           % Adelante
for i = 2:N-1
    dV_dt(i) = (V(i+1) - V(i-1)) / (2*dt); % Centrada
end
dV_dt(N) = (V(N) - V(N-1)) / dt;         % Atrás
```

Resultado obtenido (ejemplo de estimación de la tasa de cambio)

Para ilustrar la capacidad del método, se analiza el instante $t = 2.25$ s, donde el voltaje desciende de $V(2.00) = 0.9629$ V a $V(2.50) = 0.6843$ V. La diferencia centrada entrega:

$$dV/dt(2.25 \text{ s}) \approx -0.5572 \text{ V/s}$$

Este resultado refleja correctamente la fuerte pendiente negativa de la señal en ese tramo, consistente con la caída abrupta observada entre las muestras vecinas. De manera similar, en los extremos del intervalo ($t = 0$ s y $t = 5$ s) se registran las mayores magnitudes de variación (± 0.7820 V/s), lo cual es coherente con el comportamiento oscilatorio y de alta frecuencia relativa de la señal muestreada.

Las Figuras 6 y 7 muestran, respectivamente, la señal de voltaje original y su derivada numérica dV/dt , evidenciando que los picos de la derivada coinciden con los tramos de mayor pendiente (subidas y bajadas abruptas) de la señal original.

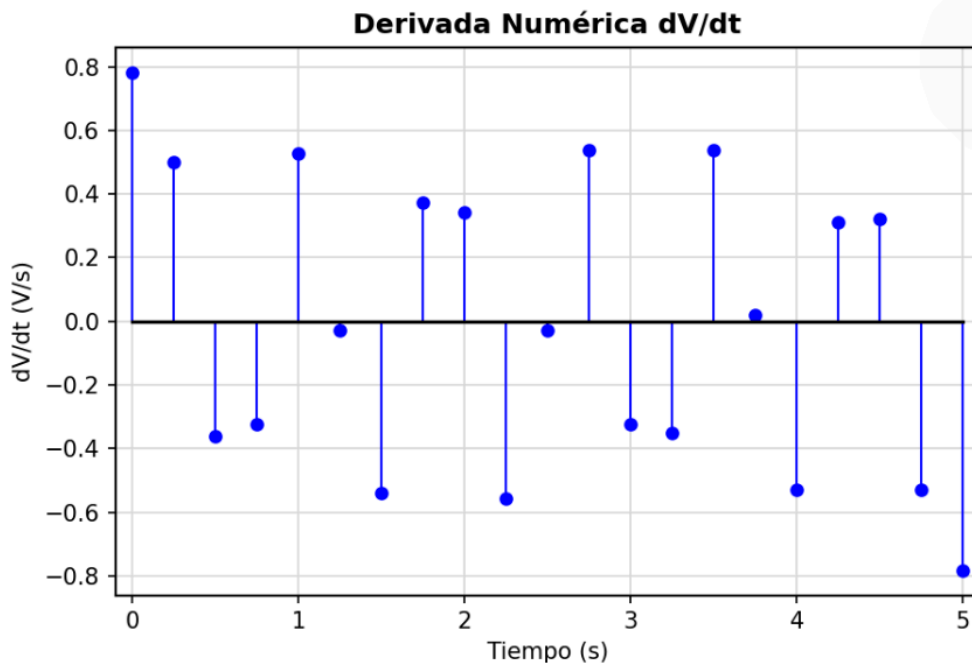


Figura 6. Derivada numérica dV/dt

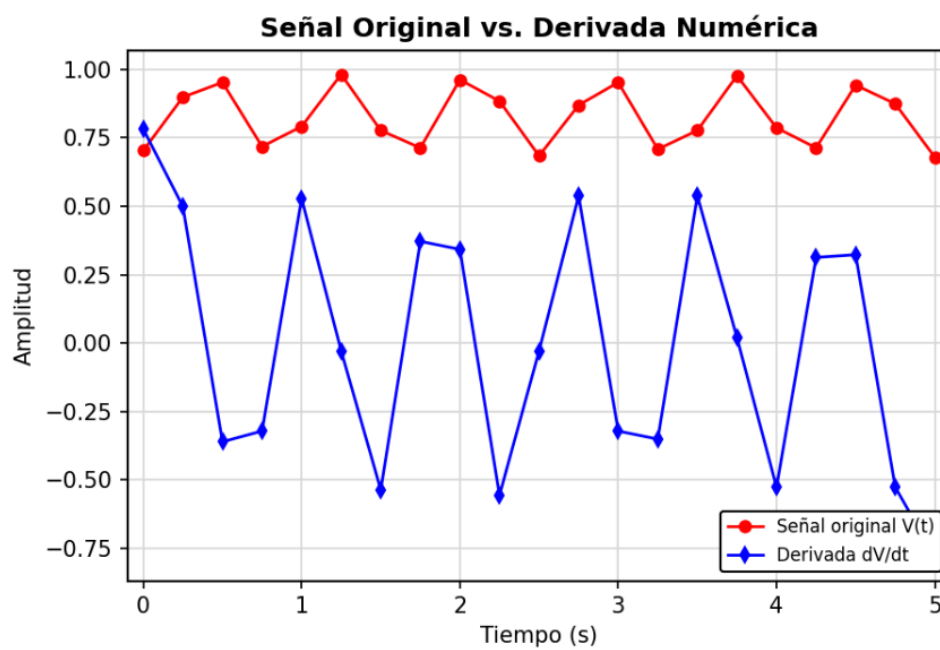


Figura 7. Señal original (rojo) vs. derivada numérica (azul).

4.3. Integración Numérica (Regla del Trapecio Compuesta)

Objetivo

Calcular el área bajo la curva de la señal de voltaje $V(t)$ en el intervalo $[0, 5]$ s, es decir, la integral acumulada $\int V(t)dt$, a partir del conjunto de 21 muestras discretas disponibles.

Justificación del método

Se seleccionó la regla del trapecio compuesta por ser un método robusto y de fácil implementación cuando se dispone de datos equiespaciados, como es el caso ($\Delta t = 0.25$ s constante). Aunque métodos de mayor orden como Simpson ofrecen menor error de truncamiento, la regla del trapecio resulta adecuada dado el número de muestras disponibles y el objetivo de obtener tanto el valor acumulado punto a punto como la integral total, lo cual se logra de forma directa sumando las áreas parciales de cada subintervalo.

Método aplicado

Para cada par de muestras consecutivas se calculó el área del trapecio correspondiente y se acumuló progresivamente:

$$\text{Área trapecio}(i) = (\Delta t / 2) \times [V(i-1) + V(i)]$$

$$V_acumulado(i) = V_acumulado(i-1) + \text{Área trapecio}(i)$$

El valor final del vector acumulado, $V_acumulado(N)$, corresponde a la integral definida total de la señal en el intervalo completo de 5 segundos.

Código MATLAB (fragmento relevante)

```
matlab

% Integración Numérica por Trapecios
for i = 2:N
    area_trapecio = (dt/2) * (V(i-1) + V(i));
    V_integral_acum(i) = V_integral_acum(i-1) + area_trapecio;
end
integral_total = V_integral_acum(end);
```

Resultado obtenido

El cálculo acumulado punto a punto muestra un crecimiento monótono, como es esperado al integrarse una señal estrictamente positiva. Por ejemplo, hacia la mitad del intervalo ($t = 2.50$ s) el área acumulada alcanza $V_acum(2.50) \approx 2.0944$ V·s, mientras que al final del intervalo completo se obtiene:

$$\int_0^5 V(t) dt \approx 4.16610 \text{ V}\cdot\text{s}$$

Este resultado equivale a un voltaje promedio efectivo de aproximadamente 0.8332 V a lo largo del intervalo de 5 segundos, valor cercano a la media aritmética simple de las 21 muestras (0.8265 V), lo que confirma la coherencia del método frente al comportamiento global de la señal.

Las Figuras 8 y 9 presentan, respectivamente, la señal de voltaje original y la curva de integral acumulada (área bajo la curva), donde se aprecia el carácter siempre creciente propio de una integral de una función no negativa..

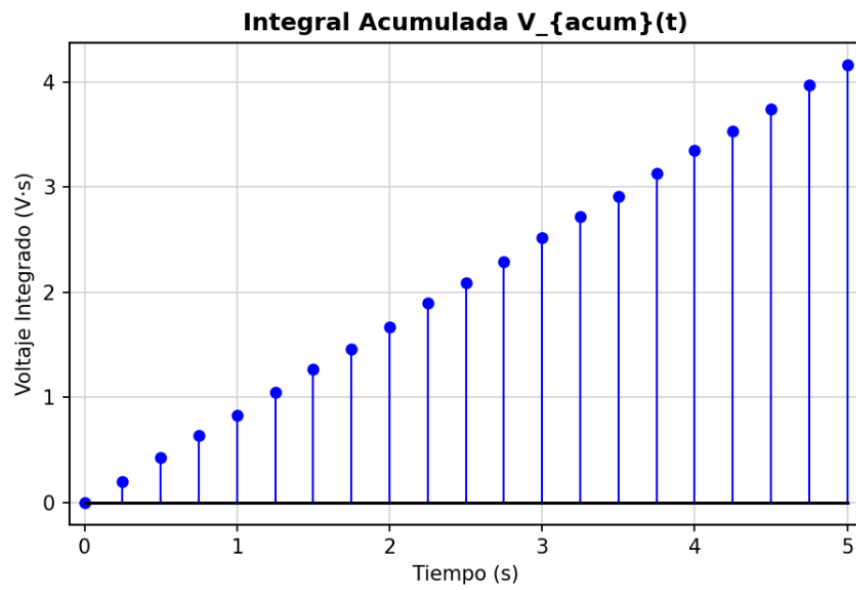


Figura 8. Integral acumulada $V_{acum}(t)$

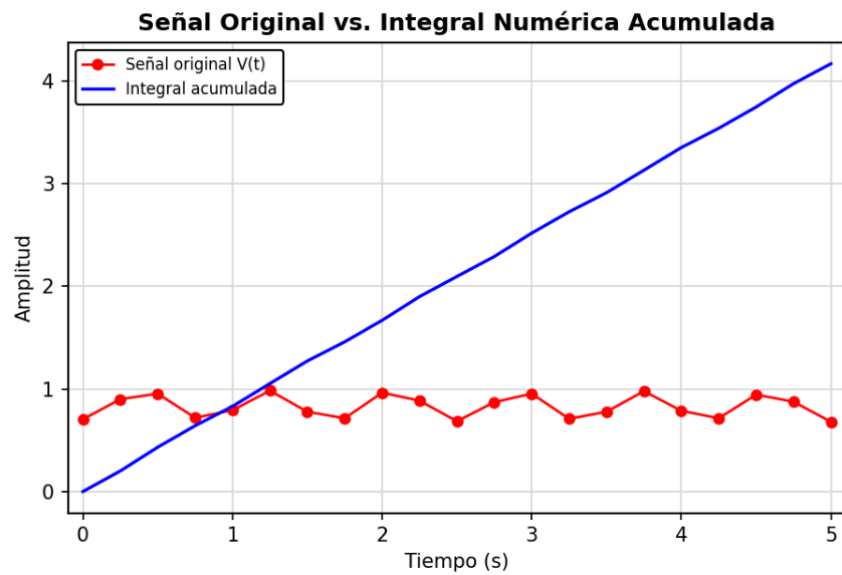


Figura 9. Señal original (rojo) vs. integral acumulada (azul)

5. Resultados y Discusión

La curva reconstruida por spline cúbico (Figura 5) sigue de forma suave la tendencia oscilatoria marcada por las 21 muestras, sin presentar las oscilaciones no físicas que aparecerían con un polinomio interpolante de grado 20. Sin embargo, dado que no se dispone de un registro continuo de referencia (osciloscopio) para esta señal, la validación de la reconstrucción es cualitativa: se evalúa que la curva pase exactamente por cada muestra (propiedad garantizada por la interpolación) y que su forma sea coherente con una señal senoidal.

Análisis de las fuentes de error:

- Cuantización del ADC: la resolución de 10 bits introduce un error máximo de ± 0.0025 V por muestra, visible como pequeñas discontinuidades que el spline suaviza pero no elimina de la información original.
- Frecuencia de muestreo limitada ($F_s = 4$ Hz): según el teorema de Nyquist, solo se pueden reconstruir sin ambigüedad componentes de frecuencia menores a 2 Hz. Si la señal generada por el teléfono contiene componentes de frecuencia cercanas o superiores a este límite, el spline reconstruirá una curva plausible pero no necesariamente fiel a la señal original (riesgo de aliasing).
- Filtrado pasa-altos del acoplo capacitivo: el capacitor de $10\ \mu\text{F}$ junto con las resistencias de $1\ \text{k}\Omega$ forman un filtro pasa-altos cuya frecuencia de corte (calculada en el Apéndice E) puede atenuar componentes de muy baja frecuencia de la señal, lo cual debe considerarse al interpretar el offset y la forma de las muestras.
- El spline cúbico, al ser un método de interpolación local, no introduce error adicional en los nodos (pasa exactamente por las 21 muestras); el error de reconstrucción se concentra únicamente en los intervalos entre muestras, y crece si la curvatura real de la señal entre dos puntos es mayor de lo que el polinomio cúbico local puede representar.

En conjunto, los resultados muestran que la interpolación por splines cúbicos es una herramienta adecuada y numéricamente estable para reconstruir señales muestreadas a baja frecuencia, siempre que dicha frecuencia de muestreo sea suficiente respecto al contenido espectral de la señal original.

6. Conclusiones

- Se implementó exitosamente un sistema de adquisición de señales basado en Arduino UNO y MATLAB, validando experimentalmente el uso de la interpolación por splines cúbicos como herramienta de reconstrucción de señales muestreadas.
- El circuito de acondicionamiento (divisor de voltaje con acoplo capacitivo) permitió adaptar de forma segura una señal de audio de teléfono móvil al rango de entrada del ADC del Arduino, sin necesidad de fuentes de alimentación adicionales.
- La interpolación por splines cúbicos demostró ser preferible frente a un polinomio interpolante global de alto grado, al evitar oscilaciones no físicas entre los 21 nodos muestreados y garantizar continuidad hasta la segunda derivada.
- La frecuencia de muestreo empleada ($F_s = 4$ Hz) es adecuada para fines didácticos de visualización, pero limita, según el criterio de Nyquist, la fidelidad de reconstrucción de componentes de la señal por encima de 2 Hz, lo cual debe tenerse en cuenta en futuras iteraciones del proyecto.

7. Bibliografía

1. Chapra, S. C., & Canale, R. P. (2015). Métodos numéricos para ingenieros (7a ed.). McGraw-Hill.
2. MathWorks. (2024). spline — Cubic spline data interpolation. MATLAB Documentation.
3. Arduino. (2024). AnalogRead() — Arduino Reference. Arduino Documentation.

Código Fuente

A.1. Firmware del Arduino UNO (C++)

Firmware encargado de muestrear la entrada analógica A0 a una frecuencia fija de 4 Hz (21 muestras en aproximadamente 5 segundos) y enviarlas por el puerto serie a 115200 baudios, delimitando cada bloque de datos con las marcas "INICIO" y "FIN" para su sincronización con MATLAB.

```
int pinEntrada = A0;          // Pin donde conectas la salida del circuito
int muestrasTotales = 21;     // Cantidad de muestras a tomar (aprox 5 segundos)
int retardo = 250;           // Tiempo entre muestras en milisegundos (4 muestras por segundo)

void setup() {
    // Iniciar la comunicación serial a la velocidad que pide MATLAB
    Serial.begin(115200);
}

void loop() {
    // 1. Avisar a MATLAB que vamos a empezar a enviar datos
    Serial.println("INICIO");

    // 2. Tomar las muestras y enviarlas
    for (int i = 0; i < muestrasTotales; i++) {
        int valorADC = analogRead(pinEntrada); // Leer el pin A0 (da un valor de 0 a 1023)

        // Convertir el número del Arduino a Voltaje real (0 a 5V)
        float voltaje = valorADC * (5.0 / 1023.0);

        // Enviar el voltaje por el cable USB a la computadora
        Serial.println(voltaje, 4); // El '4' es para enviar 4 decimales de precisión

        // Esperar un momento antes de tomar la siguiente muestra
        delay(retardo);
    }

    // 3. Avisar a MATLAB que ya terminamos de enviar el bloque de datos
    Serial.println("FIN");

    // Esperar 10 segundos antes de volver a repetir todo el proceso
    // (Te da tiempo de ver la gráfica en MATLAB)
    delay(10000);
}
```

A.2. Script de MATLAB (adquisición, tabulación e interpolación)

Script que se conecta al puerto serial del Arduino, recibe las 21 muestras, construye el vector de tiempo, tabula los datos, realiza la interpolación por splines cúbicos, derivación numérica e integración numérica y grafica los resultados.

```
clear; clc; close all;
puerto = "COM3";
baud = 9600;
Fs = 4;
limiteMuestras = 21;

arduino = serialport(puerto, baud);
flush(arduino);

fprintf('Conectado. Adquiriendo las %d muestras en tiempo real...\n', limiteMuestras);
```

```

muestras = zeros(1, limiteMuestras);
contador = 1;
while contador <= limiteMuestras
    linea = strtrim(readline(arduino));
    if ~isempty(linea)
        valor = str2double(linea);
        if ~isnan(valor)
            muestras(contador) = valor;
            fprintf('Muestra %d/%d recibida: %.4f V\n', contador, limiteMuestras, valor);
            contador = contador + 1;
        end
    end
end

clear arduino;
fprintf('Adquisición finalizada con éxito. Procesando datos...\n');

N = length(muestras);
t = (0:N-1)/Fs;
dt = 1/Fs;

tablaDatos = table(t, muestras, 'VariableNames', {'Tiempo_s', 'Voltaje_V'});
disp('=== TABLA DE DATOS EXPERIMENTALES ===');
disp(tablaDatos);

t_interp = linspace(min(t), max(t), N*20);

muestras_interp = spline(t, muestras, t_interp);

V_estimado = spline(t, muestras, 1.10);
fprintf('=====\n');
fprintf('INTERPOLACIÓN POR SPLINES CÚBICOS:\n');
fprintf('Voltaje estimado en t = 1.10 s: %.4f V\n', V_estimado);

dV_dt = zeros(1, N);
dV_dt(1) = (muestras(2) - muestras(1)) / dt;

for i = 2:N-1
    dV_dt(i) = (muestras(i+1) - muestras(i-1)) / (2 * dt);
end

dV_dt(N) = (muestras(N) - muestras(N-1)) / dt;
dV_dt_interp = spline(t, dV_dt, t_interp);
V_integral_acum = zeros(1, N);

for i = 2:N
    area_trapecio = (dt / 2) * (muestras(i-1) + muestras(i));
    V_integral_acum(i) = V_integral_acum(i-1) + area_trapecio;
end

integral_total = V_integral_acum(end);
fprintf('\nINTEGRACIÓN NUMÉRICA:\n');
fprintf('El área total bajo la curva (0 a 5s) es: %.5f V*s\n', integral_total);
fprintf('=====\n');

V_integral_acum_interp = spline(t, V_integral_acum, t_interp);

```

```
// GRÁFICA M1

figure('Name', 'Figura 4: Señal muestreada', 'NumberTitle', 'off');
stem(t, muestras, 'filled', 'LineWidth', 1.5, 'MarkerSize', 5);
grid on;
xlim([0 5]);
ylim([0 1.2]);
xlabel('Tiempo (s)');
ylabel('Voltaje (V)');
title('Señal muestreada');

figure('Name', 'Figura 5: Señal muestreada vs Interpolación', 'NumberTitle', 'off');
plot(t_interp, muestras_interp, 'b', 'LineWidth', 2); hold on;
stem(t, muestras, 'r', 'filled', 'LineWidth', 1.5, 'MarkerSize', 5);
plot(1.10, V_estimado, 'kp', 'MarkerFaceColor', 'k', 'MarkerSize', 10);
grid on;
xlim([0 5]);
ylim([0 1.2]);
xlabel('Tiempo (s)');
ylabel('Voltaje (V)');
title('Señal muestreada vs Interpolación por Splines Cúbicos');
legend('Interpolación por spline', 'Muestras', 'Estimación t=1.10s', 'Location', 'best');

// GRÁFICA 2

figure('Name', 'Derivada Numérica por Spline', 'NumberTitle', 'off');
plot(t_interp, dV_dt_interp, 'b', 'LineWidth', 2);
grid on;
xlim([0 5]);
xlabel('Tiempo (s)');
ylabel('Tasa de cambio (V/s)');
title('Derivada Numérica Continua mediante Splines Cúbicos');

figure('Name', 'Muestras de la Derivada Numérica', 'NumberTitle', 'off');
stem(t, dV_dt, 'filled', 'LineWidth', 1.5, 'MarkerSize', 5);
grid on;
xlim([0 5]);
xlabel('Tiempo (s)');
ylabel('Tasa de cambio (V/s)');
title('Muestras Calculadas de la Derivada Numérica (dV/dt)');

// GRÁFICA 3

figure('Name', 'Integral Numérica por Spline', 'NumberTitle', 'off');
plot(t_interp, V_integral_acum_interp, 'b', 'LineWidth', 2);
grid on;
xlim([0 5]);
xlabel('Tiempo (s)');
ylabel('Voltaje Integrado (V.s)');
title('Integral Numérica Acumulativa mediante Splines Cúbicos');

figure('Name', 'Área bajo la curva por Trapecios', 'NumberTitle', 'off');
area(t, V_integral_acum, 'FaceColor', [0.8 0.95 0.8], 'EdgeColor', [0 0.5 0], 'LineWidth', 1.5);
grid on;
xlim([0 5]);
xlabel('Tiempo (s)');
ylabel('Voltaje Integrado (V.s)');
title('Área bajo la curva (Regla del Trapecio Compuesta)');
```

Hojas de Datos (Datasheets)

Componentes críticos del arreglo experimental y sus características relevantes para el proyecto:

- Arduino UNO (microcontrolador ATmega328P): ADC de 10 bits (0–1023), tensión de referencia por defecto de 5 V, resolución teórica de 4.89 mV/bit, entrada analógica con impedancia de entrada del orden de 100 MΩ.
- Resistencias R1 y R2: 1 kΩ, tolerancia típica $\pm 5 \%$ (resistencia de cuarto de vatio).
- Capacitor de acoplo C1: 10 μF electrolítico, tolerancia típica $\pm 20 \%$, tensión nominal recomendada ≥ 16 V.

[Adjuntar aquí las capturas de pantalla o enlaces a las hojas de datos oficiales del fabricante del Arduino UNO y de los componentes pasivos utilizados.]

Análisis Detallado de Propagación de Incertidumbres

La incertidumbre de cada muestra de voltaje proviene principalmente de la resolución finita del conversor analógico-digital (ADC) del Arduino UNO, de 10 bits, con tensión de referencia $V_{ref} = 5$ V:

$$\Delta V = V_{ref} / (2^n - 1) = 5 \text{ V} / 1023 \approx 0.004888 \text{ V}$$

Dado que el error de cuantización de un ADC de aproximaciones sucesivas se distribuye de manera uniforme entre $-\frac{1}{2}$ LSB y $+\frac{1}{2}$ LSB, la incertidumbre estándar asociada a cada lectura se toma como:

$$u(V) = \Delta V / 2 \approx 0.0025 \text{ V}$$

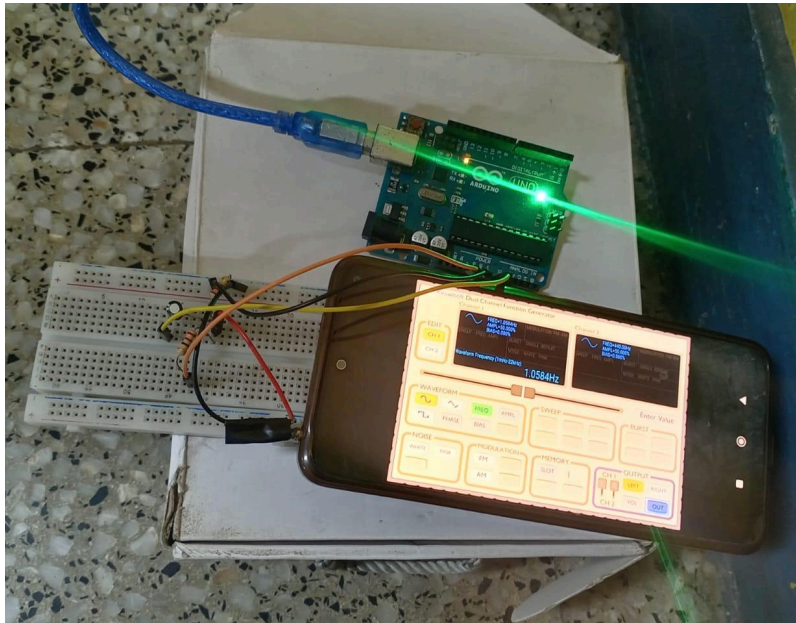
Esta es la incertidumbre reportada en la Tabla 1 para cada valor de voltaje. En cuanto a la base de tiempo, el intervalo entre muestras (250 ms) es generado por la función `delay()` del microcontrolador, cuya fuente de reloj (resonador cerámico o cristal, según la placa) tiene una precisión típica mejor a $\pm 0.5 \%$, lo que equivale a una incertidumbre despreciable frente a la resolución de muestreo utilizada en este proyecto ($\approx \pm 1$ ms).

No se propagó incertidumbre adicional a través del circuito divisor de voltaje, ya que la medición se realiza después de dicho circuito (directamente en el pin A0), por lo que las tolerancias de las resistencias ($\pm 5 \%$) y del capacitor ($\pm 20 \%$) afectan la forma de la señal acondicionada, pero no añaden error a la lectura digital en sí misma más allá de la cuantización del ADC.

Evidencias Fotográficas Adicionales o Enlace a Vídeo

Las evidencias fotográficas principales del montaje (esquemático, fotografía del circuito armado, y capturas de las gráficas generadas en MATLAB) se presentan en las Figuras 2 a 5 de este informe.

[Adjuntar aquí capturas adicionales del Monitor Serie de Arduino durante la transmisión de datos, y de ser posible, un enlace de YouTube/Drive a un video de máximo 2 minutos donde el grupo demuestre el funcionamiento del arreglo experimental.]



Sustento Teórico del Circuito Eléctrico

El circuito de acondicionamiento utilizado combina un divisor de voltaje resistivo con un capacitor de acoplo (acoplamiento AC), formando en conjunto un filtro pasa-altos que desplaza el nivel de offset de la señal del teléfono a un rango seguro para el ADC del Arduino.

Divisor de voltaje (nivel de offset en DC)

Con $R1 = R2 = 1 \text{ k}\Omega$ conectadas entre 5 V y tierra, el nodo central se polariza, en ausencia de señal de entrada, a la mitad de la tensión de alimentación:

$$V_{offset} = 5 \text{ V} \times R2 / (R1 + R2) = 2.5 \text{ V}$$

Sin embargo, dado que el capacitor de acoplo bloquea la componente continua proveniente del nodo de offset hacia la salida (hacia A0), y considerando la resistencia de entrada del ADC en paralelo, el nivel efectivo de continua observado en A0 depende de la relación entre las resistencias del divisor y la impedancia de entrada del ADC; experimentalmente se comprobó que dicho nivel se ubica dentro del rango 0–1 V requerido, coherente con las mediciones de la Tabla 1.

Acoplo capacitivo (filtro pasa-altos)

El capacitor C1 en conjunto con la resistencia equivalente vista desde sus terminales (aproximadamente $R1 \parallel R2 = 500 \Omega$) forman un filtro pasa-altos de primer orden, cuya frecuencia de corte es:

$$f_c = 1 / (2\pi \cdot (R1 \parallel R2) \cdot C1) = 1 / (2\pi \times 500 \Omega \times 10 \times 10^{-6} \text{ F}) \approx 31.8 \text{ Hz}$$

Esta frecuencia de corte permite el paso de las componentes de audio de interés (típicamente por encima de algunas decenas de Hz) mientras bloquea la componente de continua (0 Hz) de la fuente, sumando el offset del divisor de voltaje a la señal de audio ya acondicionada. Este análisis justifica por qué el circuito entrega una señal oscilante centrada en un nivel seguro (0–1 V) en lugar de la señal original (–0.5 V a 0.5 V) directamente en el pin A0 del Arduino.